This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

ResFed: Communication Efficient Federated Learning with Deep Compressed Residuals

Rui Song, *Member, IEEE,* Liguo Zhou, Lingjuan Lyu, *Member, IEEE,* Andreas Festag, *Senior Member, IEEE,* and Alois Knoll, *Fellow, IEEE*

***Abstract*—Federated learning allows for cooperative training among distributed clients by sharing their locally learned model parameters, such as weights or gradients. However, as model size increases, the communication bandwidth required for deployment in wireless networks becomes a bottleneck. To address this, we propose a residual-based federated learning framework (ResFed) that transmits residuals instead of gradients or weights in networks. By predicting model updates at both clients and the server, residuals are calculated as the difference between updated and predicted models and contain more dense information than weights or gradients. We find that the residuals are less sensitive to an increasing compression ratio than other parameters, and hence use lossy compression techniques on residuals to improve communication efficiency for training in federated settings. With the same compression ratio, ResFed outperforms current methods (weight- or gradient-based federated learning) by over $1.4\times$ on federated datasets, including MNIST, FashionMNIST, SVHN, CIFAR-10, CIFAR-100, FEMNIST, in client-to-server communication, and can also be applied to reduce communication costs for server-to-client communication.**

***Index Terms*—federated learning, communication efficiency, deep compression, protocol design**

I. Introduction

FEDERATED learning has become an emerging machine learning paradigm, which enables distributed training on broad data sources without disclosing their original data [1]. Instead of transmitting raw data, only parameters (mostly model weights or gradients) in federated learning are iteratively shared between clients and a server via heterogeneous networks. Federated learning has been successfully applied to various applications [2], such as mobile keyboard prediction, speech recognition, image object detection, etc.

However, with the increasing size of machine learning models, the existing mobile communication infrastructure cannot always meet the requirement in terms of bandwidth and latency in federated learning, which limits the wide deployment of federated learning. For instance, to train a transformer model with billions of parameters, usually 32-bit float parameters, the size of a message in a single federated learning round can be several 10 or 100 Gigabytes, e.g., a CTRL model [3] with 1.6 billion parameters or a T5 model [4] with up to 11 billion parameters. This can cause enormous and extremely costly data traffic, even in 5G NR networks, where the throughput can be from 5 Gbps to 18 Gbps. Another application scenario is to improve machine learning models for road traffic object recognition and detection in V2X (Vehicle-to-Everything) communication networks [5], where the bandwidth for V2X is also occupied for other traffic services at the same time, e.g. collective perception service, and obviously the safety-related services should have higher priority. Therefore, communication efficiency is a pivotal component for deploying federated learning, especially in wireless networks.

In an attempt to tackle the communication bottleneck, the parameter compression is considered as one of the most effective approaches, which allows for updating the models by transmitting much smaller size of messages in networks, and thereby reduces the required time per communication round in federated learning. The approaches proposed by [6]–[8] can effectively reduce the communication volume in each round by various quantization techniques, however they only consider the communication efficiency for uploading (client-to-server) but not for downloading (server-to-client). [9] compress the gradients instead of the model weights for distributed learning, which can not well fit federated learning, where clients can train multiple epochs in each round. [10] and [11] use knowledge or dataset distillation to learn and transmit a more compact model or synthetic dataset, where the original local model structure or distribution can be affected. Furthermore, most of those works attempt to compress the model parameters or gradients based on the model in a specific round, without consideration of inter-round model update similarity, which contains additional redundancy sequentially.

Inspired by residuals in video compression protocols by [12], we introduce a residual-based federated learning framework, termed as *ResFed*. It allows the server and clients to share and update models by sharing model residuals rather than model weights or gradients. Particularly, by observing training trajectory in each local client and the aggregation trajectory in the server, we believe model updates in clients and the server can be predictable. Those predictive models – in analogy with predictive frames in video transmission – can foresee model updates in the federated learning. After each communication round, we use the deviations between the predictive and the

This work was supported by the German Federal Ministry for Digital and Transport (BMVI) in the projects "KIVI – KI im Verkehr Ingolstadt" and "5GoIng – 5G Innovation Concept Ingolstadt".

Rui Song and Andreas Festag are with Fraunhofer Institute for Transportation and Infrastructure Systems IVI, Ingolstadt, Germany, e-mail: {rui.song, andreas.festag}@ivi.fraunhofer.de.

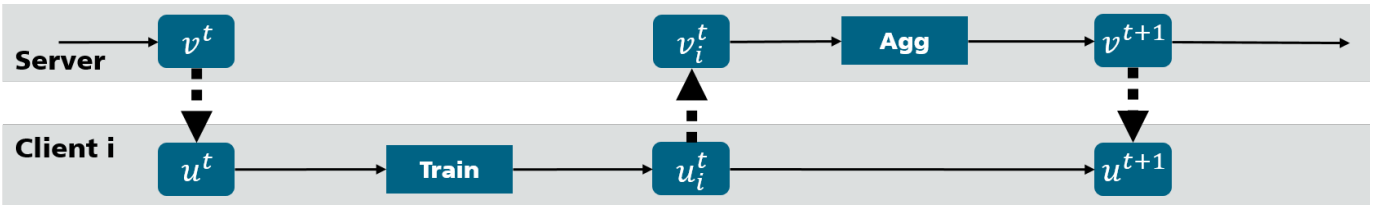
Rui Song, Liguo Zhou and Alois Knoll are with the Chair of Robotics, Artificial Intelligence and Real-Time Systems, Technical University of Munich, Garching, Germany, e-mail: {rui.song, liguo.zhou}@tum.de, knoll@in.tum.de.

Lingjuan Lyu is with Sony AI, Tokyo, Japan, e-mail: lingjuan.lv@sony.com.

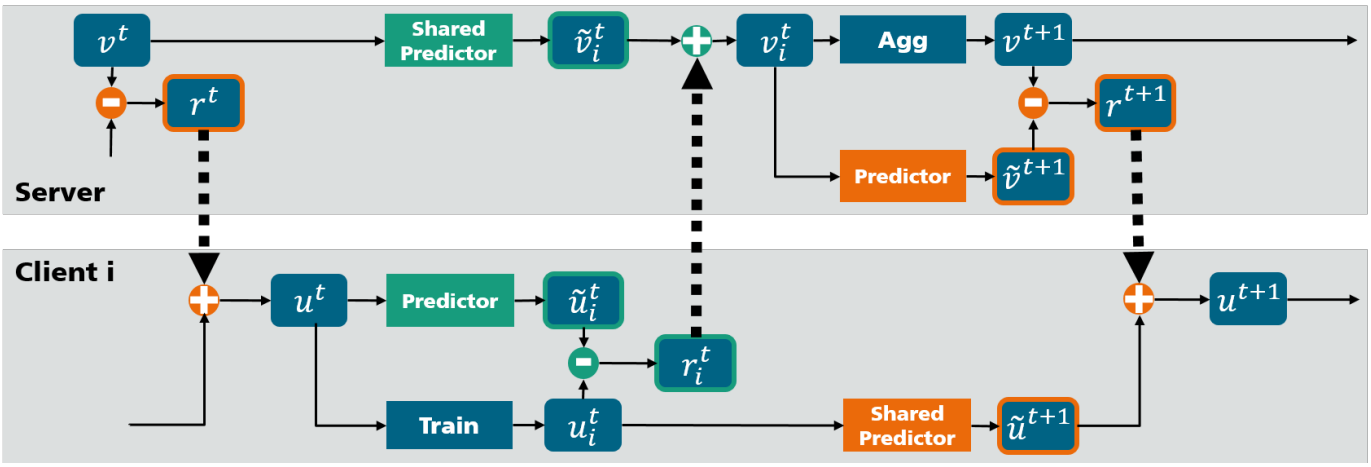
Andreas Festag is with Technische Hochschule Ingolstadt, CARISSMA Institute for Electric, COnnected, and Secure Mobility (C-ECOS), Ingolstadt, Germany, e-mail: andreas.festag@carissma.eu.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079



(a) Standard federated learning system



(b) Residual-based federated learning system

Figure 1: Paradigm shift from standard federated learning to residual-based federated learning system, with additional two pairs of predictors and corresponding operators (in green and orange). The model is updated in the client i by local training (Train) and in the server by aggregation (Agg).

actual updated model parameters, which we call the model residuals, for the communication in networks. Note that the actual updated models can be always recoverable by acquiring the residuals, as the predictors in the senders are shared to the receivers in *ResFed*. More details are provided in Sec. III and IV.

Unlike transmitting model weights, *ResFed* can wring out the potential redundancy by removing the predictive information from history updates and only keep the residuals for communication. Compared to transmitting model gradients after each training epoch, *ResFed* allows the models to be trained locally multiple times. Compared to transmitting residual accumulation for multiple epochs, *ResFed* further minimizes the information by predicting the model updates from history. As shown in Fig. 2, the values of residuals are overall smaller than weights and gradients during the entire training process. To further shrink the size of messages for communication, we then compress only residuals using sparsification and quantization, and encode the messages for information sharing in client-to-server and server-to-client.

Our main contributions are summarized as follows:

- We introduce and formulate the model residuals for the communication efficiency in federated learning and indicate the residuals contain more dense information than model weights and gradients.
- We propose a novel federated learning framework, called *ResFed*, which leverages deep residual compression to improve communication efficiency. The framework consists of the following steps: predictor sharing, model prediction, residual generation, residual compression, residual communication, model recovery, and model trajectory synchronization.
- We provide a comprehensive evaluation of our framework under varying communication cost constraints, in both up- and down-stream communication, giving insight into its viability for deployment in resource-constrained communication environments. Our open-source implementation of *ResFed* is made publicly accessible in GitHub [1].

II. Related Work

Deep compression. Deep compression was originally proposed by [13] and aims at compressing deep learning models by a pipeline including sparsification (pruning), quantization, and encoding for more efficient deployment. Based on deep compression, [9] have proposed deep gradient compression to reduce communication costs in distributed learning by compressing gradients rather than model weights, which can also be used in federated learning.

However, models are usually trained more than one epoch locally in federated learning [1], [14]–[16], which results in gradient accumulation instead of gradients in other distributed learning scenarios [17]. Our empirical experiments in Sec. VI-B also show compressing residuals can achieve better communication efficiency compared to compressing gradients due to the additional prediction step. In *ResFed*, we especially consider

[1] https://github.com/rruisong/ResFed

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

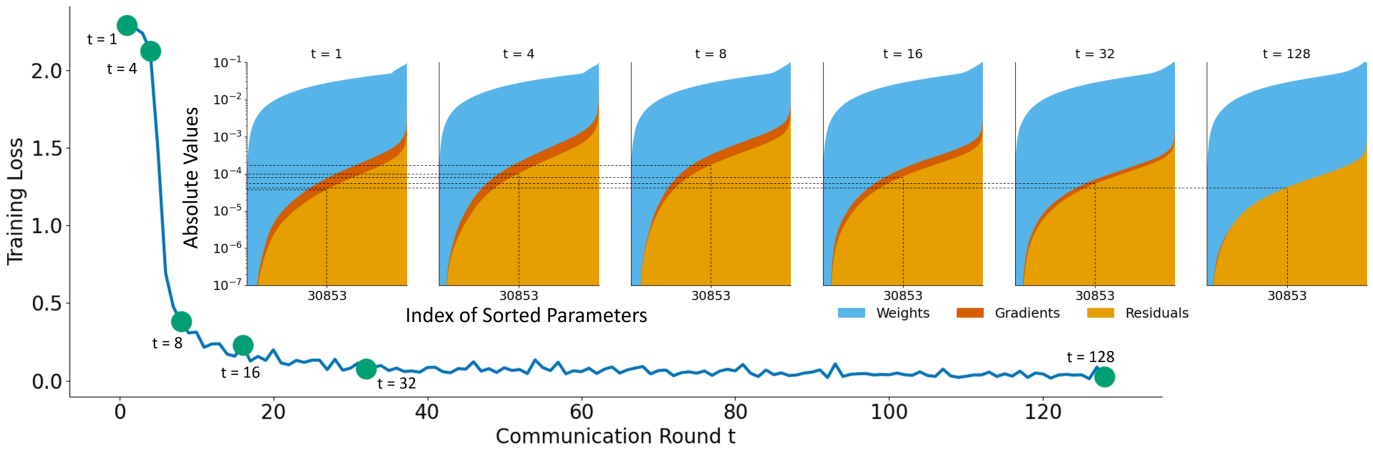


Figure 2: Value comparison of model parameters, gradients and residuals in federated learning. We train a LeNet with $61-706$ weights of 32-bits float on the MNIST datasets distributed among 10 clients, with fixed learning rate 0.001 and batch size 64. To fairly compare the gradients and residuals, the number of local epochs in each client is set as 1. We set 6 checkpoints when the number of communication rounds is $\{1,4,8,16,32,128\}$. The results show that most of the residual values are smaller than the weights and gradients during training. This suggests that lossy compression of residuals tends to preserve more information and has less impact on the accuracy.

residuals, which eliminate the model similarity in a single inter-round of federated learning communication and achieve a better compression performance by leveraging the deep residual compression.

Federated learning and communication efficiency. Communication efficiency is the key for deploying federated learning in real application scenarios, especially to train a large model. Previous research by [16], [18]–[21] attempted to reduce the number of communication rounds needed for better communication efficiency. Meanwhile, the approaches proposed by [6]–[8], [22] are built upon deep compression and focus on improving communication efficiency by decreasing communication volume. However, unlike compressing residuals in *ResFed*, they compressed model weights without considering any potential redundancy in sequential updating of federated learning. The recent work by [23] has also mentioned the update of the predictive model in federated learning, which is concurrent with our work, but the information in the history of model update is not considered to reduce the redundancy of the parameters.

Additionally, all those algorithms above can only be used to improve communication efficiency for up-streaming, while *ResFed* can handle with both up- and down-streaming for heterogeneous resource-constrained environments.

Residuals in video encoding. The residuals have been widely and successfully utilized in video encoding since H.261 [12], [24]. By considering inter-frame correlations, the pixel values in the current frame are predicted from history frames and then only residuals, i.e. the deviations between predicted and the actual pixel values in the current frame, are encoded and streamed to the receivers. Inspired by the residuals in video encoding, we integrate the model residuals into federated learning in *ResFed*, where the inter-round similarity of a model update is analogous to inter-frame correlation in video encoding.

III. Model Residual and Compression

To set the foundation for our framework, we begin by presenting the relevant concepts and methods that are utilized. Our focus is on the communication between a single client i and the server in a federated learning system that comprises N clients and one server, as shown in Fig 1. The information exchange occurs from communication round t to $t+1$ and is consistent for all other clients.

Notation. Throughout this paper, we adopt the following notations: w represents the model weights, r denotes the model residuals. To distinguish between the parameters in clients and the server, v is used to represent the model weights on the server, while u stands for the model weights on the client side. The residuals of client i during uploading are represented as $r^t_{i,ul}$ and the residuals of the server during downloading are denoted as $r^t_{i,dl}$. We provide an overview of the most relevant notations in Tab. I.

A. Model Update

Client. Given a client i with a local dataset $\mathscr{D}_i$, the initial local model in round t is u^{t-1}_i. The client first receives the global model v^t from the server and updates its local model to $\hat{u}^t_i$, represented as $u^{t-1}_i \rightarrow \hat{u}^t_i$. The local model $\hat{u}^t_i$ is then trained on $\mathscr{D}_i$ to produce the updated model u^t_i, represented as $\hat{u}^t_i \rightarrow u^t_i$. Note that the first updated model $\hat{u}^t_i$ is equal to or similar to the global updated model v^t, i.e. $\hat{u}^t_i \simeq u^t$. If communication involves lossy compression, there may be loss that cannot be repaired, resulting in $\hat{u}^t_i \sim u^t$.

Server. The global model v_{t-1} on the server undergoes two updates after one communication round t. The first update occurs when it receives models from the clients, represented as $v^{t-1} \rightarrow \{\hat{v}^t_i | i = 1,2,...,N\}$. The models are then aggregated to produce the second update $\{\hat{v}^t_i | i = 1,2,...,N\} \rightarrow v^t$.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

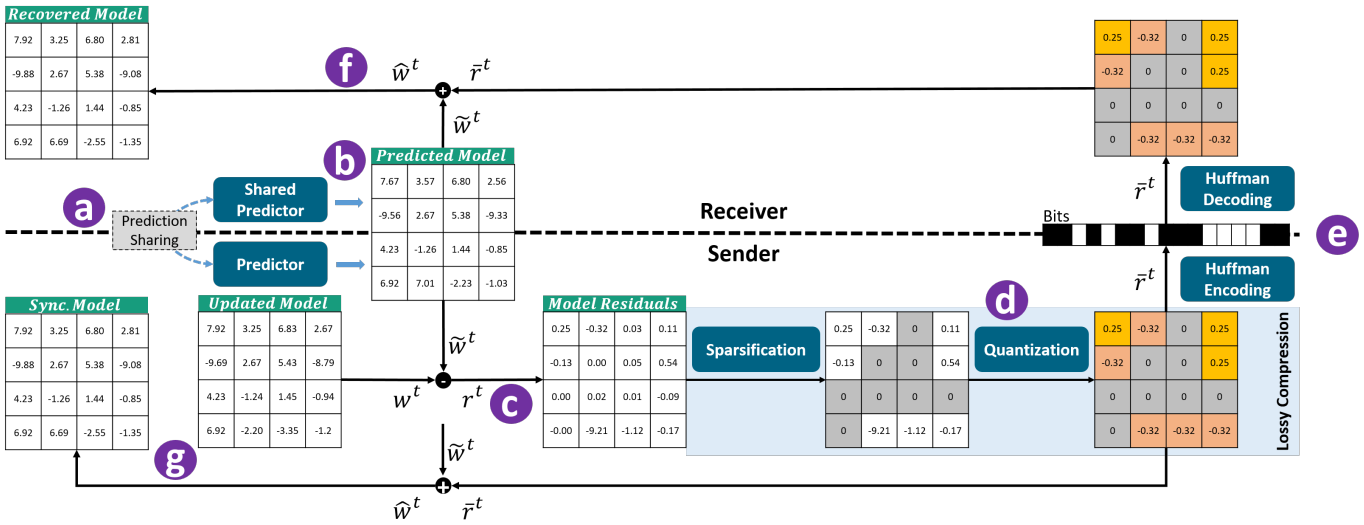


Figure 3: *ResFed* system model with a lossy deep compression pipeline for efficiently transmitting encoded model residuals. The following steps should be implemented for one communication: *(a)* Share the predictor for both sender and receiver before the federated learning starts; *(b)* Execute the same model prediction before communicating any model information; *(c)* Generate the model residuals; *(d)* Compress residuals using deep compression; *(e)* Communicate residual bits with encoding and decoding; *(f)* Recover the model from received residuals; *(g)* Synchronize the model trajectory by simulation, recovering the model locally with consideration of lossy compression.

B. Model Trajectory

Client. Consider a client i at time point t, and let us cache the updated models with a sliding time window of $[t-M,t]$ in two separate queues that distinguish the two model updates. We refer to the sequence of local model updates $\mathscr{L}_i^t = \{u_i^{t-M},...,u_i^t\}$, from $u_i^t \to \hat{u}_i^t$, as the local model trajectory, and to the sequence of global model updates $\hat{\mathscr{G}}_i^t = \{\hat{u}_i^{t-M},...,\hat{u}_i^t\}$, from $\hat{u}_i^t \to u_i^{t+1}$, as the global model trajectory.

Server. Similarly, at the server, we cache the local and global model updates for all clients in the local and global model trajectories, respectively, i.e. $\{\hat{\mathscr{L}}_i | i = 1,2,...,N\}$ and $\{\mathscr{G}_i | i = 1,2,...,N\}$. Note that if the server can always send lossless global model updates to all clients, the global trajectories at time t would be the same for all clients.

C. Model Prediction

Client. Given a client i at time point t, we predict $\hat{u}_i^t \to u_i^t$ from the local and global training trajectories, $\mathscr{L}_i^{t-1}$ and $\hat{\mathscr{G}}_i^{t-1}$ as follows:

$$\begin{aligned}\tilde{u}_i^t &= f_{predict,i}(\mathscr{L}_i^{t-1}, \hat{\mathscr{G}}_i^{t-1}, \hat{u}_i^t) \\ &= \underset{u_i^t}{arg\,max} P(u_i^t | \underbrace{u_i^{t-M},...,u_i^{t-1}}_{\mathscr{L}_i^{t-1}}, \underbrace{\hat{u}_i^{t-M},...,\hat{u}_i^{t-1}}_{\hat{\mathscr{G}}_i^{t-1}}, \hat{u}_i^t),\end{aligned} \quad (1)$$

where $f_{predict,i}$ is the used predictor for model prediction in the client i.

Server. For the server, we predict model updates $\hat{v}_i^t \to v^t$ for each client i from local and global trajectories $\hat{\mathscr{L}}_i^{t-1}$ and $\mathscr{G}_i^{t-1}$ as follows:

$$\begin{aligned}\tilde{v}_i^t &= h_{predict,i}(\hat{\mathscr{L}}_i^{t-1}, \mathscr{G}_i^{t-1}, \hat{v}_i^t) \\ &= \underset{v_i^t}{arg\,max} P(v_i^t | \underbrace{v_i^{t-M},...,v_i^{t-1}}_{\hat{\mathscr{L}}_i^{t-1}}, \underbrace{\hat{v}_i^{t-M},...,\hat{v}_i^{t-1}}_{\mathscr{G}_i^{t-1}}, \hat{v}_i^t),\end{aligned} \quad (2)$$

where $h_{predict,i}$ is the used predictor for model prediction in the server for each client i.

D. Model Residual

Given a model update $v^{t-1} \to \hat{v}_i^t$ for the client i at time t in the server or $\hat{u}_i^t \to u_i^t$ in the client i, if we can compute the model prediction $\tilde{v}_i^t$ or $\tilde{u}_i^t$ based on (1), we define the model residual as follows:

$$r_{i,ul}^t = u_i^t - \tilde{u}_i^t, \quad or \quad r_{i,dl}^t = v_i^t - \tilde{v}_i^t, \quad (3)$$

where $r_{i,ul}^t$ is the residuals from the client i for uploading, and $r_{i,dl}^t$ is the residuals from the server for downloading, respectively.

E. Model Compression

To reduce the model size for more efficient communication, we shrink the model size before sending it out. We define a compressed model in the client i:

$$\bar{u}_i^t = f_{compress}(u_i^t), \quad (4)$$

and in the server:

$$\bar{v}_i^t = h_{compress}(v_i^t), \forall i \in \{1,...,N\}, \quad (5)$$

where $f_{compress}$ and $h_{compress}$ is the used compressor for model compression in clients and server respectively. In our system,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

Algorithm 1 : *ResFed*: Residual-based federated learning framework. The sections highlighted in indicate the model recovery module, and the sections highlighted in indicate the module for residual computation and compression.

server initializes the global model v, the empty local model trajectories $\hat{\mathscr{L}}_1,...,\hat{\mathscr{L}}_N$, the global model trajectories $\mathscr{G}_1,...,\mathscr{G}_N$, the predictor $h_{predict}$
for $i \in \{1,2,...,N\}$ **do**
    client i initializes an empty local model trajectories $\mathscr{L}_i$, an empty global model trajectories $\hat{\mathscr{G}}_i$
    $h'_{predict,i} = h_{predict}$ ▷ share predictors to client i
    $f'_{predict,i} = f_{predict,i}$ ▷ share predictors to server
end for
for $t \in \{1,2,...,T\}$ **do**
    for $i \in \{1,2,...,N\}$ **in parallel do**
        if $t < M$ **then**
            $\hat{\mathscr{G}}_i \leftarrow cache(v)$ ▷ cache the global model
            server communicates w to the client i
            $\hat{v}_i \leftarrow$ **LocalTrain**(v)
            client i communicates $\hat{w}_i$ to the server
            $\mathscr{L}_i \leftarrow cache(\hat{v}_i)$ ▷ cache the local model
        else
            server communicates $\bar{r}_{i,dl}$ to the client i
            ResFedClientUpdate $(i, \bar{r}_{i,dl})$
            client i communicates $\bar{r}_{i,ul}$ to the server
            $\tilde{v}_i \leftarrow f'_{predict,i}(\mathscr{G}_i, \hat{\mathscr{L}}_i, \hat{v})$
            $\hat{v}_i \leftarrow \tilde{v}_i + \bar{r}_{i,ul}$
            $\hat{\mathscr{L}}_i \leftarrow cache(\hat{v}_i)$
        end if
    end for
    $v \leftarrow$ **Aggregate**$(\hat{v}_1,...,\hat{v}_N)$
    for $i \in \{1,2,...,N\}$ **do**
        $\tilde{v}_i \leftarrow h_{predict}(\mathscr{G}_i, \hat{\mathscr{L}}_i, v)$
        $r_{i,dl} \leftarrow v - \tilde{v}_i$
        $\bar{r}_{i,dl} \leftarrow h_{compress}(r_{i,dl})$
        $\mathscr{G}_i \leftarrow cache(\tilde{v}_i + \bar{r}_{i,dl})$
    end for
end for
return v

ResFedClientUpdate $(i, \bar{r})$
    $\tilde{u} \leftarrow h'_{predict,i}(\hat{\mathscr{G}}_i, \mathscr{L}_i, u_i)$
    $\hat{u} \leftarrow \tilde{u} + \bar{r}$
    $\hat{\mathscr{G}}_i \leftarrow cache(\hat{u})$
$u_i \leftarrow$ **LocalTrain**$(\hat{u})$
    $\tilde{u}_i \leftarrow f_{predict,i}(\hat{\mathscr{G}}_i, \mathscr{L}_i, u_i)$
    $r_i \leftarrow u_i - \tilde{u}_i$
    $\bar{r}_i \leftarrow f_{compress}(r_i)$
    $\mathscr{L}_i \leftarrow cache(\tilde{u}_i + \bar{r}_i)$
return $\bar{r}_i$

we consider to compress and communicate model residuals instead of model itself in the client i:

Table I: Summary of mainly used notations.

Notation	Meaning	Sec.
i	Index of clients	III-A
t	Index of communication round	III-A
η	Learning rate	IV-A
N	Number of connected clients	III
T	Number of total rounds	III-B
V	Number of parameters in a machine learning model	VII
w	Model weights[1]	III-A
w^t	Model weights in round t	III-A
w_i	Model weights in client i	III-A
$\bar{w}$	Compressed model weights	III-E
$\tilde{w}$	Predicted model weights	III-C
$\hat{w}$	Received model weights for update	III-A
u_i	Model weights on client i	III-A
$\bar{u}_i$	Compressed model weights on client i	III-E
$\tilde{u}_i$	Predicted model weights on client i	III-C
$\hat{u}_i$	Received global model weights from the server for update on client i	III-A
v	Model weights on the server	III-A
v^t	Model weights in round t on the server	III-A
v_i	Model weights for client i on the server	III-A
$\bar{v}$	Compressed model weights on the server	III-E
$\tilde{v}$	Predicted model weights on the server	III-C
$\hat{v}_i$	Received local model weights from client i for update on the server	III-A
g	Model gradients	IV-A
$\tilde{g}$	Predicted model gradients	V-C
r	Model residuals	III-D
r^t	Model residuals in round t	III-D
r_i	Model residuals in client i	III-D
$r_{i,dl}$	Model residuals of client i for uploading in client i	III-D
$r_{i,ul}$	Model residuals of server for downloading in client i	III-D
$\bar{r}$	Compressed model residuals	III-E
$\hat{r}$	Received model residuals	III-A
$\mathscr{D}$	Dataset	III-A
$\mathscr{G}$	Global trajectory queue	III-B
$\mathscr{L}$	Local trajectory queue	III-B

[1] To distinguish between the parameters in clients and the server, v is used to represent the model weights on the server, while u stands for the model weights on the client side.

$$\begin{aligned}\bar{r}^t_{i,ul} &= f_{compress}(r^t_{i,ul}) \\ &= f_{compress}(u^t_i - f_{predict,i}(u^{t-M}_i,...,u^{t-1}_i,\hat{u}^{t-M}_i,...,\hat{u}^{t-1}_i,\hat{u}^t_i)),\end{aligned} \tag{6}$$

and in the server:

$$\begin{aligned}\bar{r}^t_{i,dl} &= h_{compress}(r^t_{i,dl}) \\ &= h_{compress}(v^t_i - f_{predict,i}(v^{t-M}_i,...,v^{t-1}_i,\hat{v}^{t-M}_i,...,\hat{v}^{t-1}_i,\hat{v}^t_i))\end{aligned} \tag{7}$$

IV. ResFed: Residual-based Federated Learning Framework

The overview of the *ResFed* is shown in Fig. 1 and the detailed steps with lossy compression in one communication round is given in Fig. 3. In particular, we introduce *(a)* predictor sharing, *(b)* model prediction, and *(c)* residual generation in Sec IV-A. In Sec. III-E, *(d)* residual compression is formulated in Eq. 6 and Eq. 7. Then, after *(e)* communicating residual bits, we provide details on *(f)* model recovering in Sec. IV-B. Finally in Sec. IV-C, we describe *(g)* trajectory synchronization.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

A. Predictor Sharing

In our *ResFed* framework, a pair of predictors is deployed in both clients and the server, enabling the execution of model predictions based on the local and global model trajectories over time. The server caches the local model trajectories of all clients once local model updates are received. When the received models in the server match both the local model and the models in $\hat{\mathscr{G}}_i$ for client i, both of the client's trajectories are considered fully observable in the server.

The predictor on client i, $f_{predict,i}$, is shared with the server, resulting in $f'_{predict,i} = f_{predict,i}$. If the trajectories are fully observable in the server, the same model predictions can be made by both the server and the clients using Eq. 1. The model updates at time $t+1$ can then be recovered by communicating the model residuals and combining them with the model of the last time point. Similarly, the set of predictors on the server $\{h_{predict,i} | i = 1,...,N\}$ is shared with the corresponding client, resulting in $h'_{predict,i} = h_{predict,i}, \forall i \in 1,...,N$, allowing the same model predictions to be made using Eq. 2.

In this work, the predictor design is chosen based on the model transition dynamics within a sliding history time window. The predictor is formulated as follows:

$$\begin{aligned}\tilde{w}_i^t &= f_{predict}(\mathscr{L}_i^{t-1}, \hat{\mathscr{G}}_i^{t-1}, \hat{w}_i^t) \\ &= \begin{cases} \hat{w}_i^t, & M = 0, \\ \hat{w}_i^t + & \\ \sum_{\tau=1}^{M} (-1)^{M-\tau}(M-\tau+1)(w_i^{t-\tau} - \hat{w}_i^{t-\tau}), & M > 0. \end{cases}\end{aligned} \tag{8}$$

To reduce memory usage for caching trajectories in clients, we apply a short time window $[t-M, t]$ in the prediction process. We categorize the predictors into two types: *(i)* the stationary predictor when $M = 0$, and *(ii)* the linear predictor when $M = 1$. Note that we consider the model updates introduced by [25]–[27] as special residuals, which are calculated using the stationary predictor. In Fig. 4, we compare the statistical properties (sum and variance) of the residual values generated by the stationary and linear predictors. The residual values generated by the linear predictor have a lower sum and a more concentrated variance compared to the stationary predictor at the start of federated learning (before convergence), potentially achieving higher accuracy with the same compression ratio, as shown in Fig. 5.

Specifically, the stationary predictor predicts the next model by using the current model, $\tilde{w}_i^t = \hat{w}_i^t$, resulting in a model residual of $r_i^t = w_i^t - \hat{w}_i^t$. When the number of local training epochs is fixed to 1, the stationary residuals become proportional to gradients, i.e. $r = \eta g$, where η is the learning rate and g represents the gradients. The linear predictor, on the other hand, takes into account the model transition in the last local training step, $\tilde{w}_i^t = \hat{w}_i^t + w_i^{t-1} - \hat{w}_i^{t-1}$, resulting in a model residual of $r_i^t = w_i^t - \hat{w}_i^t - w_i^{t-1} + \hat{w}_i^{t-1}$. The predictor for the client i on the server $h_{predict,i}$ is similar to $f_{predict,i}$.

B. Model Recovery

In this setup, we cache the model trajectories in both the server and the clients. Each client maintains a history of its local and global model updates, while the server caches the global trajectory and the local trajectories of all connected clients. By sharing the predictors, the server can obtain the same model prediction, $\tilde{w}_i^t$, as the client i at round t. If the client sends an uncompressed model residual ($\hat{w}_i^t = w^t$), the server can recover the model update after local training as follows:

$$w_i^t = r_i^t + \hat{w}_i^t = r_i^t + w^t, \tag{9}$$

where $\hat{w}_i^t$ is the global model in the previous round. Similarly, by predicting the global model update in the client and sending uncompressed residuals, the aggregated model can also be recovered in the client.

C. Model Trajectory Synchronization

In order to ensure that the model updates are consistent across all clients and the server, we need to synchronize the model trajectories. The compression applied to the model residuals results in a loss of information, meaning that the updated model of a sender cannot be fully recovered in receivers, as $\bar{r} \neq r$. This leads to a difference between the original model w in the sender and the recovered model $\hat{w} = \bar{r} + \tilde{w}$ in receivers, causing drift in the results of shared predictors.

To avoid the drift effect, we synchronize the model trajectories by simulating the recovery process in the sender. Instead of caching the original model updates, we recover the model locally from the compressed residuals, which helps ensure that the trajectories in the sender are consistent with those in the receiver. This is achieved through the *ResFed* pseudocode, as outlined in Algorithm 1.

V. Theoretical Analysis

The process of *ResFed* can be bifurcated into two distinct parts: calculating the residual during upload and download. However, this section is focused solely on the analysis of residual properties during the upload phase. The characteristics of the downloading residual are analogous to those of the upload process. If we take into account the remaining clients as a singular entity, this process can be viewed as approximately symmetric, which is experimentally validated in Fig. 6.

A. Lossless Residual

In terms of the residual, we can propose the following statement.

Definition 1. (Fully recoverability.) *For any given time $t > 0$ during the uploading process, let us consider $\hat{v}_i^t$ to represent the recovered weights for the i-th client on the server. If it holds that $\hat{v}_i^t = u_i^t$ consistently, we consider the weights for the i-th client within $\hat{v}_t^t$ to be fully recoverable on the server.*

Such a condition signifies respect towards the convergence property of gradient-based federated learning. An analogous statement applies during the download process.

Theorem 1. *If ResFed is implemented without an integrated lossy compression method, it can always be fully recoverable. As a result, the convergence of ResFed is consistent with that of gradient-based federated learning.*

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

Proof. When $t \leq T$, the weights in the client and the server are identical, i.e., $u_i^t = v_i^t$. Assuming that the predictor is shared between the client and the server, and $t = T$ the following holds:

$$\begin{aligned} \tilde{u}_i^t &= f_{predict,i}(u_i^t), \\ \tilde{v}_i^t &= f_{predict,i}(v_i^t). \end{aligned} \tag{10}$$

The residual for the i-th client, represented as $r_{i,ul}^{t+1}$, can then be calculated as $r_{i,ul}^{t+1} = u_i^{t+1} - \tilde{u}_i^t$. After this, the i-th client communicates $r_{i,ul}^{t+1}$ to the server. On the server side, the residual received is recovered by $\hat{v}_i^{t+1} \leftarrow \tilde{v}_i^t + r_{i,ul}^{t+1}$. By combining these, we come to the conclusion that $\hat{v}_i^{t+1} = u_i^{t+1}$. □

B. Assumptions and Remarks

Now we consider that the lossy compression is integrated in ResFed. Typically, in federated learning, the local objective is to reduce the average loss from all clients. In this section, we represent the loss function for each client as f_i [2].

Definition 2. (Compression error). *For lossy compression, $\forall i \in \{1,2,...,N\}$, exist an compression error ε_i between sent weight u_i and received weight $\hat{v}_i$:*

$$\varepsilon_i = \hat{v}_i - u_i. \tag{11}$$

Here, we know $\hat{v}_i - u_i = \tilde{v}_i + \bar{r}_i - \tilde{u}_i - r_i$. Because the predicted weights are the same on the sender and receiver, $\tilde{v}_i = \tilde{u}_i$, so we have:

$$\varepsilon_i = \bar{r}_i - r_i. \tag{12}$$

For a local client i, the local update is at time $t > 0$:

$$\hat{v}^t = \hat{v}^{t-1} - \frac{1}{N}\sum_{i=1}^{N}(f_{compress}(r_i^t + \varepsilon_i^t) + \eta\tilde{g}_i^t). \tag{13}$$

where $\tilde{g}_i^t$ is the inexact gradient based on prediction, $\tilde{g}_i^t = g_i^t - \frac{r_i^t}{\eta}$, which can be fully observable for both sender and receiver due to the trajectory synchronization introduced in Sec. III-B. To compare with a compression result in non-distributed learning, we also conduct the update for global gradient as follows:

$$\hat{v}^t = \hat{v}^{t-1} - f_{compress}(\frac{1}{N}\sum_{i=1}^{N}(\eta\nabla f(u_i^t) + \varepsilon_i^t)) - \frac{1}{N}\sum_{i=1}^{N}\eta\tilde{g}_i^t. \tag{14}$$

We follow the assumptions made by the previous work [28]–[30].

Assumption 1. (L-Lipschitz smoothness). *The function f_i is continuously differentiable. $\forall i \in \{1,2,...,N\}$, $\exists L$:*

$$||\nabla f(u_i^{t+1}) - \nabla f(u_i^t)|| \leq L||u_i^{t+1} - u_i^t||. \tag{15}$$

Assumption 2. (Bounded gradient). *The stochastic gradient for f are unbiased. There exists constant $G > 0$, such that $\forall i \in \{1,2,...,N\}$,*

$$\mathbb{E}[g_i] = \nabla f(u_i), \tag{16}$$

$$\mathbb{E}||\frac{1}{N}\sum_{i=1}^{N} g_i||^2 \leq G^2. \tag{17}$$

[2]Here, the loss function is denoted by f without the subscript of *compress* ($f_{compress}$) or *predict* ($f_{predict}$), differentiating it from compression and prediction operations.

where g_i is a stochastic gradient at each node.

We consider the error between compression in local and global way is small, which also introduced in previous work [31], [32]:

Assumption 3. (Bounded distributed compression error. [32]) *There exist a small constant ξ, such that:*

$$\begin{aligned} ||f_{compress}(\frac{1}{N}\sum_{i=1}^{N}(r_i^t + \varepsilon_i^t)) - \frac{1}{N}\sum_{i=1}^{N} f_{compress}(r_i^t + \varepsilon_i^t)|| \\ \leq \xi||r_i^t||. \end{aligned} \tag{18}$$

Assumption 4. (Bounded residual). *There exists a constant $1 \geq \zeta > 0$, such that:*

$$\mathbb{E}_t||r^t||^2 = \mathbb{E}_t||u^t - f_{predict}(u^t)||^2 \leq \zeta^2\mathbb{E}_t||\eta g^t||^2. \tag{19}$$

We combine the Assumption 4 and bounded gradient in Assumption 2, then get:

$$\mathbb{E}_t||r^t||^2 \leq \eta^2\zeta^2G^2. \tag{20}$$

Remark 1. *For constant residual, we have $\zeta = 1$. For linear residual, based on Assumption 1, we have:*

$$\begin{aligned} \mathbb{E}||r^t||^2 &= \mathbb{E}_t||\nabla f(u_i^t) - \nabla f(u_i^{t-1})||^2 \\ &\leq L^2\mathbb{E}||u^{t-1} - \eta g^t - u^{t-1}||^2 \\ &= \eta^2L^2G^2. \end{aligned} \tag{21}$$

Given that both inequalities (21) and (20) need to be met concurrently, we adopt the lower value on their right-hand sides as the upper bound of the residual.

Remark 2. *For linear residual, there exist a constant $R = min(\eta^2L^2G^2, \eta^2\zeta^2G^2)$, such that:*

$$\mathbb{E}||r^t||^2 \leq R^2, \tag{22}$$

Assumption 4 is considered for the residuals. For that, we experimentally validate the Assumption 4 for linear residuals in Fig. 4. We show that the residual expectations are lower than the gradient by comparing the sum of parameters in the model weight.

C. Convergence for ResFed

Now, we need to specify the concrete compression method we want to use. In our compression pipeline defined in Fig. 3, the ε_i appears due to sparsification and quantization. Here we take the Top-K sparsification method as an example for further analysis and we have the guarantee of convergence for *ResFed*.

Theorem 2. ResFed *with a sparse linear residual can converge throughout the duration of training. Given a fixed learning rate defined as $\theta/\sqrt{T}$, where $\theta > 0$ is a constant, then for any time instance $t > 0$, we have:*

$$\begin{aligned} &\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}||\nabla f(u_t)||^2 \\ &\leq \frac{4\theta^{-1}(f(\hat{v}^0) - f(\hat{v}^*)) + 2\theta LG^2}{\sqrt{T}} + \frac{4\theta L^2R^2(1+\frac{\xi}{N\gamma})^2D}{\sqrt{T}}. \end{aligned} \tag{23}$$

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

Proof. If Assumption 1 holds, we infer:

$$\begin{aligned} & f(\hat{v}^{t+1}) - f(\hat{v}^t) \\ \leq & \langle \nabla f(\hat{v}^t), \hat{v}^{t+1} - \hat{v}^t \rangle + \frac{L}{2} ||\hat{v}^{t+1} - \hat{v}^t||^2. \end{aligned} \tag{24}$$

Calculating the expectation at t, we obtain:

$$\begin{aligned} & \mathbb{E}[f(\hat{v}^{t+1})] - f(\hat{v}^t) \\ \leq & \eta \langle \nabla f(\hat{v}^t), \hat{v}^{t+1} - \hat{v}^t \rangle + \frac{L\eta^2}{2} \mathbb{E}||\hat{v}^{t+1} - \hat{v}^t||^2 \\ \leq & -\frac{\eta}{2}(||\nabla f(\hat{v}^t)||^2 + L^2 ||u^t - \hat{v}^t||^2) \\ & + \frac{L\eta^2 G^2}{2} + \eta L^2 ||u^t - \hat{v}^t||^2. \end{aligned} \tag{25}$$

Then we calculate the expectation with respect to the gradient before t, it yields.

$$\begin{aligned} & \mathbb{E}[f(\hat{v}^{t+1})] - \mathbb{E}[f(\hat{v}^t)] \\ \leq & -\frac{\eta}{2} \mathbb{E}[||\nabla f(\hat{v}^t)||^2 + L^2 ||u^t - \hat{v}^t||^2] \\ & + \frac{L\eta^2 G^2}{2} + \eta L^2 \mathbb{E}||u^t - \hat{v}^t||^2. \end{aligned} \tag{26}$$

Next, by incorporating Lemma 3 into (26):

$$\begin{aligned} & \mathbb{E}[f(\hat{v}^{t+1})] - \mathbb{E}[f(\hat{v}^t)] \\ \leq & -\frac{\eta}{2} \mathbb{E}[||\nabla f(\hat{v}^t)||^2 + L^2 ||u^t - \hat{v}^t||^2] \\ & + \frac{L\eta^2 G^2}{2} + \eta^3 (1 + \frac{\xi}{N\gamma})^2 L^2 R^2 D. \end{aligned} \tag{27}$$

Using Assumption 1, we can infer:

$$\begin{aligned} ||\nabla f(u^t)||^2 \leq & 2||\nabla f(u^t) - \nabla f(\hat{v}^t)||^2 + 2||\nabla f(\hat{v}^t)||^2 \\ \leq & 2L^2 ||u^t - \hat{v}^t||^2 + 2||\nabla f(\hat{v}^t)||^2. \end{aligned} \tag{28}$$

Taking the expectation of (28) and incorporating (27):

$$\begin{aligned} & \mathbb{E}||\nabla f(u^t)||^2 \\ \leq & \mathbb{E}[2L^2 ||u^t - \hat{v}^t||^2 + 2||\nabla f(\hat{v}^t)||^2] \\ \leq & \frac{4(\mathbb{E}[f(\hat{v}^t)] - \mathbb{E}[f(\hat{v}^{t+1})])}{\eta} \\ & + 2L\eta G^2 + 4(1 + \frac{\xi}{N\gamma})^2 L^2 G^2 D. \end{aligned} \tag{29}$$

Given a fixed learning rate defined as $\theta/\sqrt{T}$, where $\theta > 0$ is a constant, it follows that:

$$\begin{aligned} \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}||\nabla f(u_t)||^2 \leq & \frac{4(f(\hat{v}^0) - f(\hat{v}^*)}{T\eta} \\ & + 2\eta L G^2 + 4\eta L^2 R^2 (1 + \frac{\xi}{N\gamma})^2 D \\ = & \frac{4\theta^{-1}(f(\hat{v}^0) - f(\hat{v}^*)) + 2\theta L G^2}{\sqrt{T}} + \frac{4\theta L^2 R^2 (1 + \frac{\xi}{N\gamma})^2 D}{\sqrt{T}}. \end{aligned} \tag{30}$$

□

The convergence rate for *ResFed* is approximately $1/\sqrt{T}$, aligning with that of FedSGD. When employing only lossless compression methods, *ResFed*'s performance mirrors conventional federated learning, as depicted in Fig. 1b. Opting to lossy compress the residual in *ResFed*, rather than compressing gradients as in other federated learning frameworks, can potentially enhance the convergence rate. This enhancement arises from the reduced upper bound of the residual's expectation value, compared to the gradient, as shown in Figs. 2 and 4. Furthermore, due to the superior compression efficiency of the residual, ResFed can achieve a higher compression ratio while maintaining the same accuracy, as demonstrated in Figure 5. More details of the lemmas and proofs can be found in the Appendix A.

Table II: Dataset and models in experiments

Dataset	Model	# of Param	Size in MB
MNIST	LeNet-5	61706	0.25
CIFAR-10	CNN (5Conv+3FC)	1020160	4.08
Fashion-MNIST	LeNet-5	61706	0.25
SVHN	ResNet-18	11175370	44.70
CIFAR-100	CNN (5Conv+3FC)	1020160	4.08
FEMNIST	CNN (2Conv+1FC)	110526	0.44

Table III: The communication volume (CV) and the bitsaving rate (BR) to reach the target accuracy (ACC) for only using *ResFed* in uploading (UL) and downloading (DL). We use FedAvg for both baseline (without any compression) and ResFed (Res-1). Note that the compression ratio per communication round is set from 350× to 375×.

Dataset		Fashion-MNIST		CIFAR-10		SVHN	
		IID	Non-IID	IID	Non-IID	IID	Non-IID
Target ACC		85%		70%		88%	
Baseline	CV	17.73 Mb	29.55 Mb	261.16 Mb	587.61 Mb	7.15 Gb	10.73 Gb
ResFed UL	CV	0.16 Mb	0.28 Mb	4.09 Mb	6.79 Mb	0.08 Gb	0.17 Gb
	BR	**99.10%**	**99.10%**	**98.43%**	**98.84%**	**98.89%**	**98.42%**
ResFed DL	CV	0.10 Mb	0.21 Mb	1.48 Mb	4.61 Mb	0.07 Gb	0.11 Gb
	BR	**99.43%**	**99.30%**	**99.43%**	**99.22%**	**99.02%**	**98.97%**

VI. Experiment

A. Experimental Settings

Guided by the previous work by [22], [36], [37], we process and distribute the datasets MNIST [38], Fashion-MNIST [39], SVHN [40], CIFAR-10 [41], CIFAR-100 [40] and FEMNIST [36], [42] on a set of clients, and train LeNet-5, CNN (5Conv+3FC), ResNet-18 and CNN (2Conv+1FC) on those federated dataset distributively, as shown in Tab. II.

We employ a lossy compression approach that combines sparsification and quantization, as depicted in Fig. 3. The method involves sparsifying the model parameters using the Top-K approach, where sparsity is achieved by setting a certain number of parameters to zero, adopting the strategy employed in [43], [44]. To reduce the storage required for these zero parameters, we utilize a similar strategy as described in [9] and represent those zeros using a single float 16 parameter that denotes the number of zero parameters between two non-zero

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

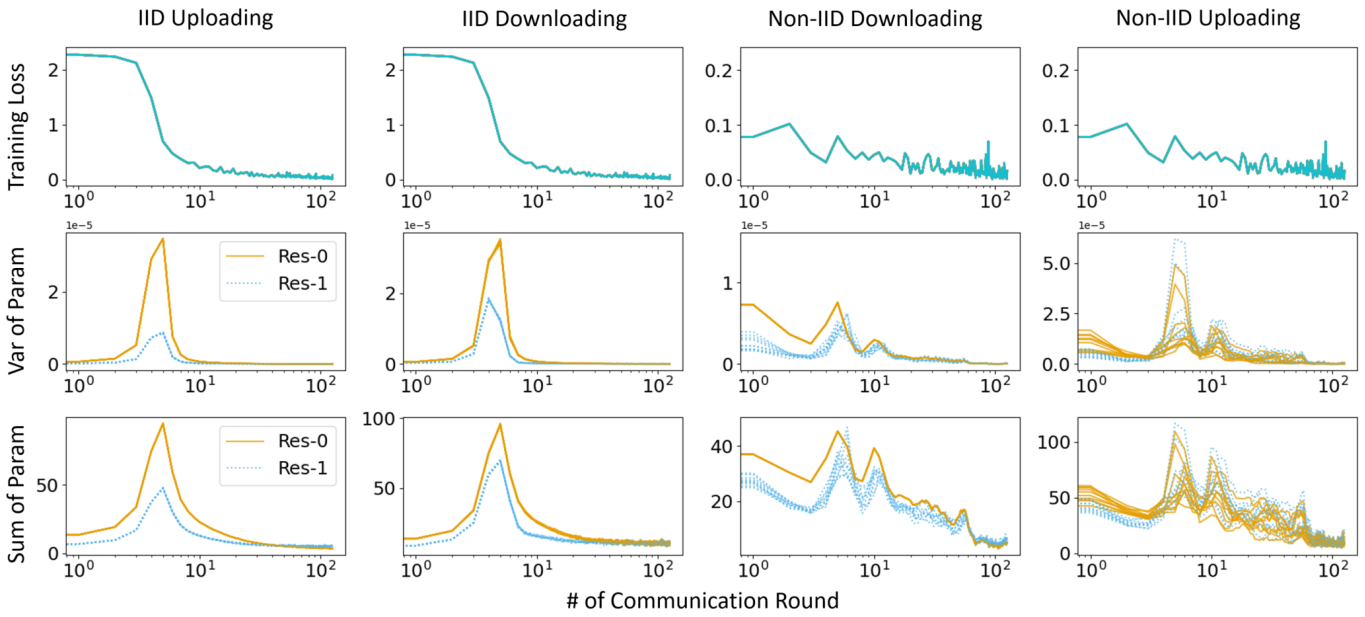


Figure 4: Comparison of linear residuals (Res-1) and stationary residuals (Res-0) while communicating in down- and uploading (server-to-client and client-to-server) for federated learning on IID and Non-IID MNIST for each of 10 clients, with fixed learning rate 0.001 and batch size 64. The number of local epochs in each client is set as 3. We show the training loss, variance (Var), and sum of parameter (Param) values. The comparison results show that the parameter values in Res-1 are lower and more concentrated than in Res-0 at the beginning of the federated learning (before convergence), which allows us to compress it with less information loss and higher accuracy using the same sparsification and quantization. The conclusion is futher validated in Fig. 5.

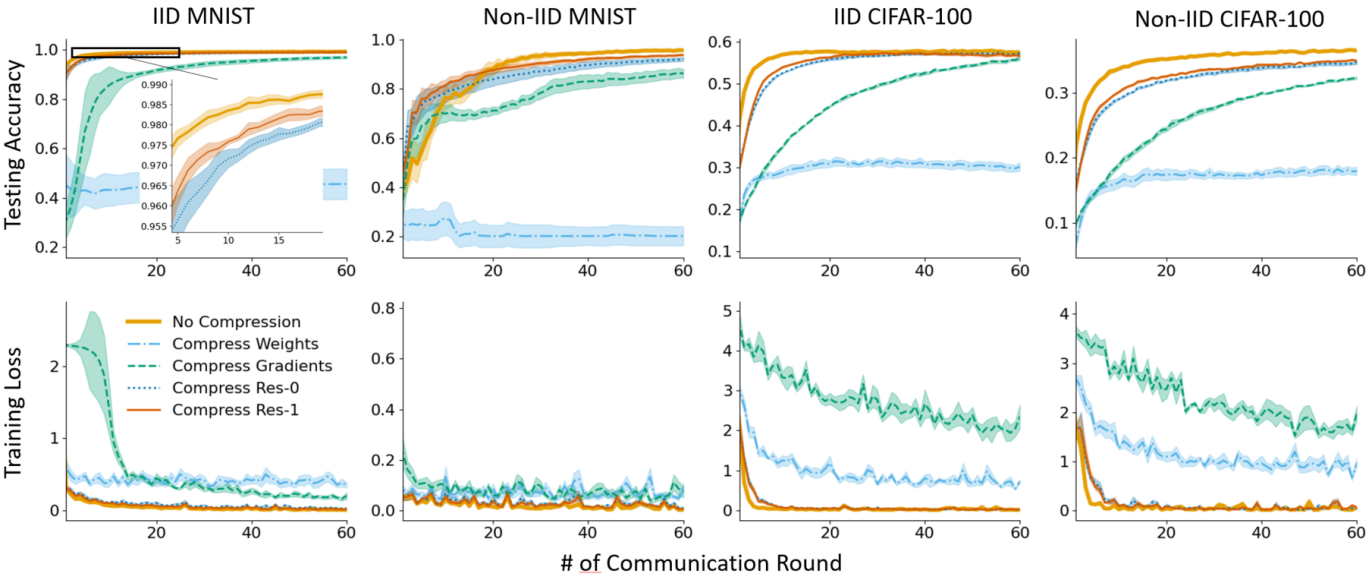


Figure 5: Comparison of compressing weights, gradients, residuals with stationary (Res-0) and linear (Res-1) predictors. The sparsities are 0.2 and 0.01 for training on MNIST and CIFAR-100 distributed in 10 clients, respectively. We quantize the non-zero parameters and use 1 bit to represent each of them. For Non-IID setting, each client owns only the data with half classes.

parameters. The remaining non-zero parameters are quantized and represented using a single bit, either 1 or -1, inspired by the approach in [6]. A codebook is created to map the 1s and -1s to their actual values. Finally, to further minimize the data size for communication, we apply Huffman encoding to all the parameters, following the technique utilized in [13].

B. Residuals vs Gradients and Weights

On top of the primary evaluation in Fig. 2, we believe deep residual compression can save more communication volume in

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

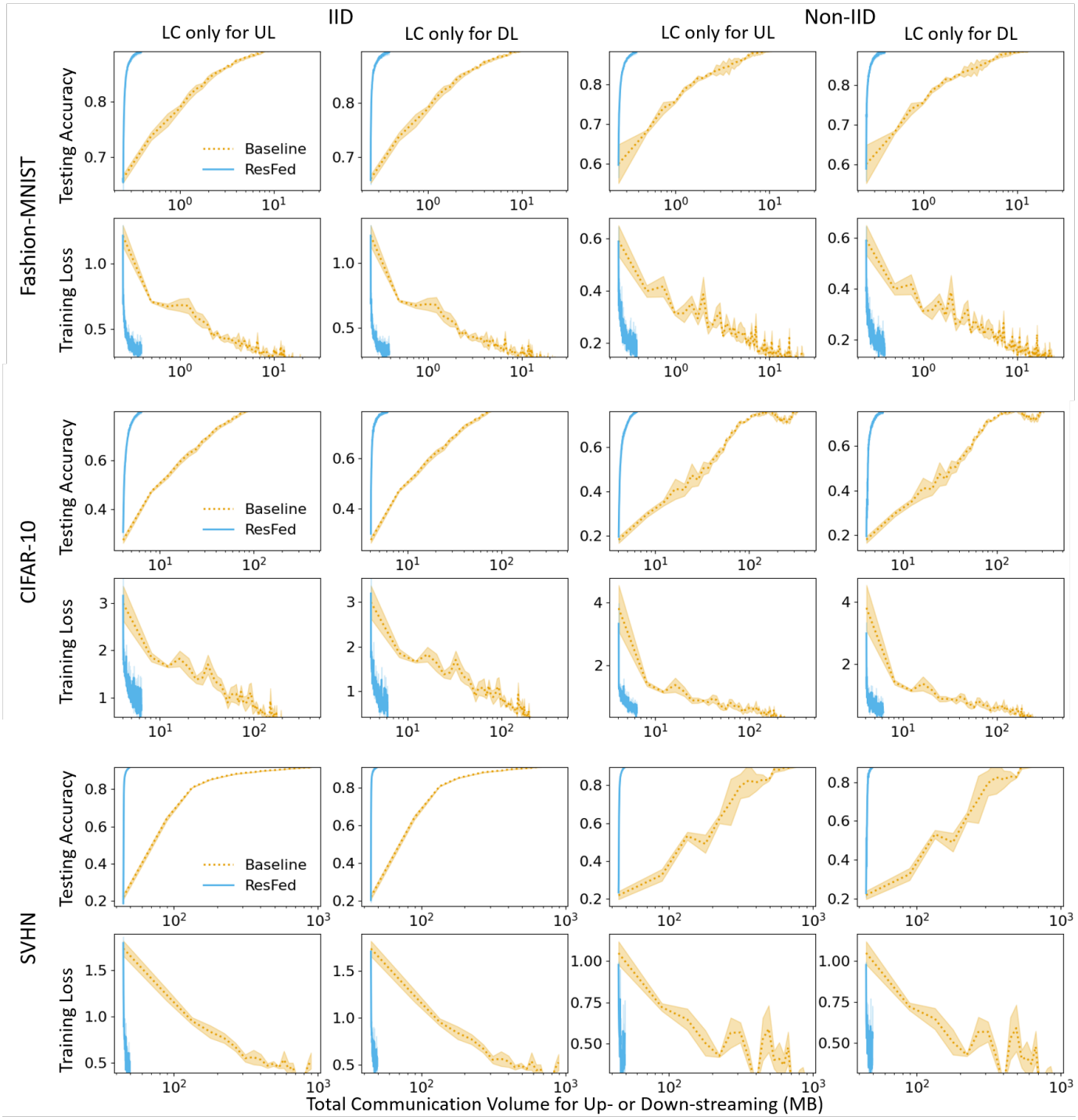


Figure 6: Overall communication efficiency enhanced by *ResFed* with Lossy Compression (LC) for only Uploading (UL) or Downloading (DL)

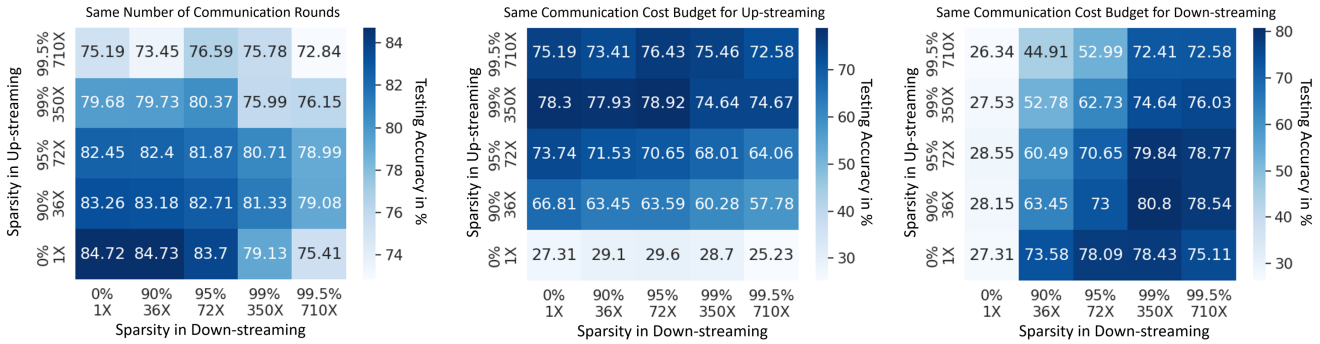
federated learning with minimum impact on the accuracy. Thus, we demonstrate the federated learning integrating compressing weights, gradients and two different residuals, i.e. stationary and linear residuals in Eq. 8. As indicated in Fig. 5, the deep residual compression approach consistently outperforms weight and gradient compression in terms of accuracy on both IID and Non-IID datasets. Moreover, the linear residuals result in a higher accuracy and faster convergence compared to the stationary residuals. These results highlight the efficacy of communicating residuals in federated learning, as it enables a higher compression ratio per communication round compared to other parameters.

C. Communication Efficiency Improvement

Next, we evaluate the required communication volume for training three sizes of models on various datasets in both IID

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079



(a) Number of communication rounds (300) is the same.

(b) The communication cost budget (14 Mb) is the same in up-streaming (left) and down-streaming (right).

Figure 7: Testing accuracy on various values of sparsity and compression ratio per communication round in deep residual compression for both up- and down- streaming in *ResFed* (Res-1). We train a CNN model on a federated CIFAR-10 dataset with 10 clients. The testing accuracy decreases with greater sparsity at the same number of communication rounds, while when the communication resource is constrained, the deep residual compression with greater sparsity can achieve more compromising testing accuracy.

Table IV: Comparison of ResFed with other advanced methods on FEMNIST with 36 clients. *Max CR (70%)* represents the maximum compression ratio achieved upon attaining 70% accuracy within 200 communication rounds, and *Max Acc (200)* signifies the peak accuracy during training.

Approach	Max CR (70%)	Max Acc (200)
FedSGD [33]	1×	80.76%
FedAvg (E=10) [1]	19×	**87.96%**
SignSGD [34]	32×	70.09%
Top-K (80%) [31]	82.93×	70.25%
1-bit CS-FL [35]	131.46×	71.02%
FTTQ [6]	-	69.18%
QSFL [22]	889.72×	78.08%
ResFed (Res-0)	192.68×	70.04%
ResFed (Res-1)	**1023.60**×	75.46%

and Non-IID settings. Tab. III shows that to reach a promising target accuracy, *ResFed* with lossy compression (compression ratio is set on $350\times - 375\times$) can save on average around 99% of the total communication volume for all clients in only up- or down-streaming. Furthermore, the bitsaving ratios of *ResFed* on IID and Non-IID settings are similar, which indicates the compression performance of *ResFed* is robust to data heterogeneity in federated learning. We show accuracy and training loss changing with increasing communication volume in Fig. 6. The results indicate communicating residuals in federated learning can remarkably save overall communication volume.

D. Comparison With Other Methods

We benchmark *ResFed* against other advanced methods focusing on communication efficiency, namely SignSGD [34], Top-K [31], 1-bit CS-FL [35], FTTQ [6] and QSFL [22]. To ensure a fair comparison, especially with the state-of-the-art method QSFL, we adopt the same FEMNIST experimental setup in [22] and consider the first 36 writers as federated learning clients for a natural data heterogeneity. We compare their maximum compression ratio after reaching a 70% accuracy over 200 communication rounds, also recording their peak accuracy within this span. We designate the communication volume in FedSGD as our baseline, thus establishing a compression ratio of 1×. Obviously, by using deep compressed residuals, *ResFed* outperforms other methods in terms of compression ratio. In particular, *ResFed* raises the compression ratio by around 15% compared to QSFL, with a marginal accuracy decline of 3.4%.

E. Scalability for Resource-constrained Communication Environments

Finally, we explore the scalability of *ResFed* by tuning compression ratios for client-to-server and server-to-client communications. This is important, as real-world applications may have heterogeneous network resources for up- and down-streaming. Fig. 7 illustrates the impact on test accuracy for various levels of sparsity, which results in different compression ratios for each communication round. From Fig. 7a, we can observe the testing accuracy reduces with higher compression ratio per communication round, when the number of communication rounds is always the same, i.e. set to 300. However, when we consider the dedicated budget for overall communication volume in up- or down-streaming, it becomes apparent that a higher compression ratio in *ResFed* results in improved accuracy due to the possibility of implementing more communication rounds, as shown in Fig. 7b. By adapting the compression ratio based on available network resources for up- or down-streaming, *ResFed* can effectively enhance federated learning through deep residual compression.

VII. Conclusion

In this work, we present a novel residual-based federated learning framework that utilizes residual sharing instead of weight or gradient sharing. The framework leverages deep

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

residual compression to enhance communication efficiency in both up- and down-streaming in federated learning, making it suitable for deployment in diverse network environments. Our experimental results demonstrate that this framework significantly reduces overall communication volume while achieving equivalent prediction accuracy compared to standard federated learning. Furthermore, *ResFed* outperforms weight or gradient compression in terms of accuracy and convergence speed.

Limitations. In our residual-based federated learning framework, local and global trajectories are stored in each client and the server for model prediction and residual calculation. When performing *ResFed* with N clients to train a model with V 32-bit float parameters, and with a trajectory length of M, each client needs $2*32*V*M$ bits of memory to store its local and global trajectories. This results in a memory requirement proportional to $M*V$ in each client. On the server side, to store the local and global trajectories for all N clients, $2*32*V*M*N$ bits of memory are needed. To reduce memory requirements, one potential solution is to cache compressed models in the trajectories of both the sender and receiver. However, this could result in reduced accuracy in model prediction, and the trade-off between memory and accuracy needs to be further explored in future work.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *International Conference on Artificial Intelligence and Statistics (AISTATS 2017)*, vol. 54, Apr. 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a/mcmahan17a.pdf

[2] L. Lyu, H. Yu, X. Ma, C. Chen, L. Sun, J. Zhao, Q. Yang, and P. S. Yu, "Privacy and robustness in federated learning: Attacks and defenses," *arXiv preprint arXiv:2012.06337*, 2020.

[3] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[4] C. Raffel *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: http://jmlr.org/papers/v21/20-074.html

[5] R. Song, L. Zhou, V. Lakshminarasimhan, A. Festag, and A. Knoll, "Federated learning framework coping with hierarchical heterogeneity in cooperative its," in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, 2022, pp. 3502–3508.

[6] J. Xu, W. Du, Y. Jin, W. He, and R. Cheng, "Ternary compression for communication-efficient federated learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 3, pp. 1162–1176, 2022, DOI: 10.1109/TNNLS.2020.3041185.

[7] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization," in *International Conference on Artificial Intelligence and Statistics (AISTATS 2020)*. PMLR, Aug. 2020, pp. 2021–2031. [Online]. Available: https://proceedings.mlr.press/v108/reisizadeh20a.html

[8] R. Hönig, Y. Zhao, and R. Mullins, "DAdaQuant: Doubly-adaptive quantization for communication-efficient federated learning," in *International Conference on Machine Learning (ICML 2022)*. PMLR, Aug. 2022, pp. 8852–8866. [Online]. Available: https://proceedings.mlr.press/v162/honig22a.html

[9] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=SkhQHMW0W

[10] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nature Communications*, vol. 13, no. 1, pp. 1–8, 2022, DOI: 10.1038/s41467-022-29763-x.

[11] R. Song, D. Liu, D. Z. Chen, A. Festag, C. Trinitis, M. Schulz, and A. Knoll, "Federated learning via decentralized dataset distillation in resource-constrained edge environments," in *2023 International Joint Conference on Neural Networks (IJCNN)*, 2023, pp. 1–10.

[12] J. Li, B. Li, and Y. Lu, "Deep contextual video compression," in *Advances in Neural Information Processing Systems (NeurIPS 20201)*, 2021, pp. 18 114–18 125. [Online]. Available: https://papers.nips.cc/paper_files/paper/2021/file/96b250a90d3cf0868c83f8c965142d2a-Paper.pdf

[13] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," *International Conference on Learning Representations (ICLR 2016)*, May 2016. [Online]. Available: https://arxiv.org/pdf/1510.00149v5.pdf

[14] T. Li *et al.*, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems (MLSys 2020)*, vol. 2, pp. 429–450, 2020. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/file/1f5fe83998a09396ebe6477d9475ba0c-Paper.pdf

[15] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 7611–7623, 2020. [Online]. Available: https://papers.nips.cc/paper_files/paper/2020/file/564127c03caab942e503ee6f810f54fd-Paper.pdf

[16] S. P. Karimireddy *et al.*, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning (ICML 2020)*, vol. 119. PMLR, Jul. 2020, pp. 5132–5143. [Online]. Available: https://proceedings.mlr.press/v119/karimireddy20a.html

[17] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Sparse binary compression: Towards distributed deep learning with minimal communication," in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–8, DOI: 10.1109/IJCNN.2019.8852172.

[18] H. Yuan and T. Ma, "Federated accelerated stochastic gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, 2020, pp. 5332–5344. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/39d0a8908fbe6c18039ea8227f827023-Paper.pdf

[19] J. Hamer, M. Mohri, and A. T. Suresh, "FedBoost: A communication-efficient algorithm for federated learning," in *International Conference on Machine Learning (ICML 2020)*. PMLR, 2020, pp. 3973–3983. [Online]. Available: https://proceedings.mlr.press/v119/hamer20a.html

[20] L. Yi, W. Gang, X. Liu, Z. Shi, and H. Yu, "FedGH: Heterogeneous Federated Learning with Generalized Global Header," in *ACM Multimedia*, 2023.

[21] Y. Tan, G. Long, L. LIU, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, "Fedproto: Federated prototype learning across heterogeneous clients," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, pp. 8432–8440, Jun. 2022. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/20819

[22] L. Yi, W. Gang, and L. Xiaoguang, "QSFL: A two-level uplink communication optimization framework for federated learning," in *International Conference on Machine Learning (ICML 2022)*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, Jul. 2022, pp. 25 501–25 513. [Online]. Available: https://proceedings.mlr.press/v162/yi22a.html

[23] K. Yue, R. Jin, C.-W. Wong, and H. Dai, "Communication-efficient federated learning via predictive coding," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 3, pp. 369–380, 2022, DOI: 10.1109/JSTSP.2022.3142678.

[24] B. Girod, E. G. Steinbach, and N. Faerber, "Comparison of the H.263 and H.261 video compression standards," in *Standards and Common Interfaces for Video Information Systems: A Critical Review*, vol. 10282. SPIE, 1995, pp. 230–248, DOI: 10.1117/12.227952.

[25] L. P. Barnes, H. A. Inan, B. Isik, and A. Özgür, "rTop-k: A statistical estimation approach to distributed SGD," *IEEE Journal on Selected Areas in Information Theory*, vol. 1, no. 3, pp. 897–907, 2020, DOI: 10.1109/JSAIT.2020.3042094.

[26] N. Mitchell, J. Ballé, Z. Charles, and J. Konečnỳ, "Optimizing the communication-accuracy trade-off in federated learning with rate-distortion theory," *arXiv preprint arXiv:2201.02664*, 2022.

[27] B. Isik, T. Weissman, and A. No, "An information-theoretic justification for model pruning," in *International Conference on Artificial Intelligence and Statistics (AISTATS 2022)*. PMLR, 2022, pp. 3821–3846. [Online]. Available: https://proceedings.mlr.press/v151/isik22a/isik22a.pdf

[28] S. Magnússon, H. Shokri-Ghadikolaei, and N. Li, "On maintaining linear convergence of distributed learning and optimization under limited communication," in *2019 53rd Asilomar Conference on Signals, Systems, and Computers*, 2019, pp. 432–436, DOI: 10.1109/IEEECONF44664.2019.9049052.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

[29] S. Shi, X. Chu, K. C. Cheung, and S. See, "Understanding top-k sparsification in distributed deep learning," *arXiv preprint arXiv:1911.08772*, 2019.

[30] S. Shi *et al.*, "Communication-efficient distributed deep learning with merged gradient sparsification on GPUs," in *IEEE Conference on Computer Communications (IEEE INFOCOM 2020)*, 2020, pp. 406–415, DOI: 10.1109/INFOCOM41043.2020.9155269.

[31] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)*. Copenhagen, Denmark: Association for Computational Linguistics, Sep. 2017, pp. 440–445, DOI: 10.18653/v1/D17-1045. [Online]. Available: https://aclanthology.org/D17-1045

[32] S. Shi, K. Zhao, Q. Wang, Z. Tang, and X. Chu, "A convergence analysis of distributed SGD with communication-efficient gradient sparsification," in *Conference on Artificial Intelligence (IJCAI 2019)*, 7 2019, pp. 3411–3417, DOI: 10.24963/ijcai.2019/473.

[33] X. Lian *et al.*, "Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/f75526659f31040afeb61cb7133e4e6d-Paper.pdf

[34] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *International Conference on Machine Learning (ICML 2018)*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 560–569. [Online]. Available: https://proceedings.mlr.press/v80/bernstein18a.html

[35] C. Li, G. Li, and P. K. Varshney, "Communication-efficient federated learning based on compressed sensing," *IEEE Internet of Things Journal*, vol. 8, no. 20, pp. 15 531–15 541, 2021, DOI: 10.1109/JIOT.2021.3073112.

[36] S. Caldas *et al.*, "LEAF: A benchmark for federated settings," *arXiv preprint arXiv:1812.01097*, 2018.

[37] T. Li, M. Sanjabi, A. Beirami, and V. Smith, "Fair resource allocation in federated learning," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=ByexElSYDr

[38] Y. LeCun and C. Cortes, "MNIST handwritten digit database," 2010, URL: http://yann.lecun.com/exdb/mnist/. [Online]. Available: https://arxiv.org/abs/1812.01097

[39] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," p. 7, Aug. 2017, arXiv:cs.LG/1708.07747.

[40] Y. Netzer *et al.*, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011.

[41] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Computer Science Department, Tech. Rep., 2009, pp. 58. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[42] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *2017 International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 2921–2926, DOI: 10.1109/IJCNN.2017.7966217.

[43] X. Ma, J. Zhang, S. Guo, and W. Xu, "Layer-wised model aggregation for personalized federated learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 092–10 101, DOI: 10.1109/CVPR52688.2022.00985.

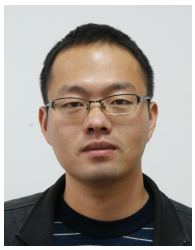
[44] S. Lu, R. Li, W. Liu, C. Guan, and X. Yang, "Top-k sparsification with secure aggregation for privacy-preserving federated learning," *Computers & Security*, vol. 124, p. 102993, 2023, DOI: 10.1016/j.cose.2022.102993.

[45] D. Alistarh *et al.*, "The convergence of sparsified gradient methods," in *Advances in Neural Information Processing Systems (NeurIPS 20218)*, vol. 31, 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/314450613369e0ee72d0da7f6fee773c-Paper.pdf

[46] J. Van Leeuwen, "On the construction of Huffman trees," in *Third International Colloquium on Automata, Languages and Programming (ICALP 1976)*, Jul. 1976, pp. 382–410. [Online]. Available: https://webspace.science.uu.nl/~leeuw112/huffman.pdf



Rui Song (Member, IEEE) received his B.S. degree in Vehicle Engineering from Hefei University of Technology (HFUT) in 2013 and M.S. degree in Mechanical Engineering from Karlsruhe Institute of Technology (KIT) in 2016. After four years as a software developer with the topic of digital validation for autonomous driving functions in industry, he started his current occupation as a researcher at Fraunhofer Institute for Transportation and Infrastructure Systems (IVI). Meanwhile, he is pursuing the Ph.D degree with the Chair of Robotics, Artificial Intelligence and Real-Time Systems of the Technical University of Munich. His research interests are federated learning, vehicular communication, cooperative perception and intelligent traffic systems.



Liguo Zhou received the Bachelor's degree from the School of Computer Science & Technology, Soochow University, Suzhou, China, in June 2012, and the Master's degree from the School of Computer Science, Wuhan University, Wuhan, China, in June 2018. He is currently pursuing the Ph.D. degree with the Chair of Robotics, Artificial Intelligence and Real-Time Systems, Technical University of Munich. His current research interests include Deep Learning, Computer Vision and Intelligent Vehicles.



Lingjuan Lyu (Member, IEEE) is a research manager and team leader in Sony AI. She received Ph.D. from the University of Melbourne. She was a winner of the IBM Ph.D. Fellowship Worldwide. Her current research interest is trustworthy AI. She had published over 50 papers in top conferences and journals, including NeurIPS, ICML, ICLR, Nature, etc. Her papers had won several best paper awards and oral presentations from top conferences.



Andreas Festag (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the Berlin Institute of Technology, in 2003. As a Researcher, he worked with the Telecommunication Networks Group (TKN), Berlin Institute of Technology; Heinrich-Hertz-Institute (HHI), Berlin; NEC Laboratories, Heidelberg; Vodafone Chair Mobile Communication Systems at the Dresden University of Technology; and the Fraunhofer Institute for Transportation and Infrastructure Systems (IVI). He is a Professor with Technische Hochschule Ingolstadt and the Center of Automotive Research on Integrated Safety Systems and Measurement Area (CARISSMA), the research and test center for vehicle safety. His research interests include architecture, design, and performance evaluation of wireless and mobile communication systems and protocols, with a focus on vehicular communication and cooperative intelligent transportation systems (ITS). He actively contributes to European standardization in ITS.



Alois Knoll (Fellow, IEEE) is a professor of computer science with the Department of Informatics of the Technical University Munich. His research interests include robotics, artificial intelligence and real-time systems. He has (co)authored more than 600 technical papers and guest-edited international journals. He initiated the First IEEE/RAS Conference on Humanoid Robots and was General and Program Chair of various IEEE conferences. He is the specialty chief editor of the Frontiers in Neurorobotics.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

APPENDIX A
LEMMAS AND PROOFS

Building on the previous Top-K studies in [32], [45], we establish the following lemmas concerning the upper bound of error introduced by Top-K.

Lemma 1. *For any local model residual $r_i \in \mathbb{R}^V$ in client i, when we sparsify the weight parameters below the top K parameters (zeros are set), it holds that*

$$||r_i - f_{topk}(r_i)||^2 \leq \gamma ||r_i||^2, \tag{31}$$

where $\gamma = \frac{V-K}{V}$.

We denote $\bar{r}_i$ as compressed residual at client i, based on Lemma 1, we have:

$$\mathbb{E}||\sum_{i=1}^{N} r_i - \sum_{i=1}^{N} \bar{r}_i||^2 \leq \gamma ||\sum_{i=1}^{N} r_i||^2. \tag{32}$$

Lemma 2. If the above assumptions hold, when Top-K method is integrated, for any $t > 0$:

$$||u^{t+1} - \hat{v}^{t+1}|| \leq \gamma ||u^t - \hat{v}^t|| + (\gamma + \frac{\xi}{N})||r^t||. \tag{33}$$

Proof. For each client performing residual compression, according to 13, we have:

$$\begin{aligned} ||u^{t+1} - \hat{v}^{t+1}|| &= ||\frac{1}{N}\sum_{i=1}^{N}((\eta(g_i^t - \tilde{g}_i^t) + \varepsilon_i^t) \\ &+ \frac{\eta}{N}\sum_{i=1}^{N}\tilde{g}_i^t + u^t - \hat{v}^t - \varepsilon^t - \frac{\eta}{N}\sum_{i=1}^{N}\tilde{g}_i^t \\ &- \frac{1}{N}\sum_{i=1}^{N} f_{compress}(\eta(g_i^t - \tilde{g}_i^t) + \varepsilon_i^t)|| \\ &= ||\frac{1}{N}\sum_{i=1}^{N}((r_i^t + \varepsilon_i^t) - \frac{1}{N}\sum_{i=1}^{N} f_{compress}(r_i^t + \varepsilon_i^t)|| \\ &= ||\frac{1}{N}\sum_{i=1}^{N}((r_i^t + \varepsilon_i^t) - f_{compress}(\frac{1}{N}\sum_{i=1}^{N}(r_i^t + \varepsilon_i^t)) \\ &+ f_{compress}(\frac{1}{N}\sum_{i=1}^{N}(r_i^t + \varepsilon_i^t)) - \frac{1}{N}\sum_{i=1}^{N} f_{compress}(r_i^t + \varepsilon_i^t)||. \end{aligned} \tag{34}$$

Given Assumption 3, we deduce:

$$\begin{aligned} ||u^{t+1} - \hat{v}^{t+1}|| &\leq ||\frac{1}{N}\sum_{i=1}^{N}(r_i^t + \varepsilon_i^t) \\ &- f_{compress}(\frac{1}{N}\sum_{i=1}^{N}(r_i^t + \varepsilon_i^t)) + \frac{\xi}{N}||r^t||. \end{aligned} \tag{35}$$

When employing the Top-K approach for residual compression, based on Lemma 1, we can infer:

$$\begin{aligned} ||u^{t+1} - \hat{v}^{t+1}|| &\leq \gamma ||r^t + u^t - \hat{v}^t|| + \frac{\xi}{N}||r^t|| \\ &= \gamma ||u^t - \hat{v}^t|| + (\gamma + \frac{\xi}{N})||r^t||. \end{aligned} \tag{36}$$

□

Lemma 3. When selecting a learning rate schedule for *ResFed* with a linear residual, such that for any iteration $t > 0$:

$$\sum_{k=1}^{t}(2\gamma^2)^k \eta \leq D, \tag{37}$$

for some constant D, we have:

$$\mathbb{E}||u^t - \hat{v}^t||^2 \leq \eta(1 + \frac{\xi}{N\gamma})^2 D L^2 R^2. \tag{38}$$

Proof. Based on 2:

$$\begin{aligned} ||u^{t+1} - \hat{v}^{t+1}||^2 &\leq (\gamma ||u^t - \hat{v}^t|| + (\gamma + \frac{\xi}{N})||r^t||)^2 \\ &\leq 2\gamma^2 ||u^t - \hat{v}^t||^2 + 2(\gamma + \frac{\xi}{N})^2 ||r^t||^2. \end{aligned} \tag{39}$$

Iterating the inequality (39) for k from 0 to t yields:

$$||u^t - \hat{v}^t||^2 \leq (1 + \frac{\xi}{N\gamma})^2 \sum_{k=1}^{t}(2\gamma^2)^k ||r^{(t-k)}||. \tag{40}$$

Applying (37) and (21) into (40), the conclusion is reached. □

APPENDIX B
EXPERIMENTAL DETAILS AND FURTHER RESULTS

In this section, we provide the details on our conducted experiments presented in Sec. VI. We run the experiments on a computer cluster with 4× NVIDIA-A100-PCIE-40GB GPUs and 4× 32-Core-AMD-EPYC-7513 CPUs. The environment is a Linux system with Pytorch 1.8.1 and Cuda 11.1.

We demonstrate the learning task on six different datasets:

- MNIST [38]: 60,000 data points in the training set and 10,000 data points in the test set. Each data point is a 28x28 gray-scale digit image, associated with a label from 10 classes.
- CIFAR-10 [41]: 50,000 data points in the training set and 10,000 data points in the test set. Each data point is a 32x32 RGB image, associated with a label from 10 classes.
- Fashion-MNIST [39]: 60,000 data points in the training set and 10,000 data points in the test set. Each data point is a 28x28 gray-scale image, associated with a label from 10 classes.
- SVHN [40]: 73,257 data points in the training set and 26,032 data points in the test set. Each data point is a 32x32 RGB digit image, associated with a label from 10 classes.
- CIFAR-100 [40]: 50,000 data points in the training set and 10,000 data points in the test set. Each data point is a 32x32 RGB image, associated with a label from 100 classes.
- FEMNIST [36], [42]: 814,255 data points from 3,500 writers. Each data point is a 28x28 gray-scale image, associated with a label from 62 classes (10 digits, 26 lowercase, 26 uppercase).

The models trained on those dataset are shown in Tab. II.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3324079

A. Experiments for Sec. VI-B

We divide the dataset, i.e. MNIST and CIFAR-100, into 10 clients and run the FedAvg with local optimizer of stochastic gradient descent (SGD) (momentum is 0.9) to train LeNet-5 and CNN (5Conv + 3FC), respectively. The learning rate is fixed to 0.01 and the batch size is 64 for all tests. The number of local epoch is 5. In Non-IID data setting, each client owns only 2 out of 10 classes in MNIST and 50 out of 100 classes in CIFAR-100.

In clients, we consider 5 different approaches as follows:

- No Compression: Standard federated learning without any compression methods is used as the baseline, which provides the best results among all the approaches when the number of communication rounds is the same.
- Compress Weights: Before communication in standard federated learning, the model weights are first compressed.
- Compress Gradients: The gradients in each epoch are compressed and communicated to the server.
- Compress Res-0: The residuals are computed by stationary predictor, i.e. Eq. 8 with $T = 0$.
- Compress Res-1: The residuals are computed using a linear predictor, i.e., Eq. 8 with $T = 1$.

For each approach on each dataset, we run 10 tests with different seeds and show the mean value and standard variance in Fig. 5.

Lossy Compression. We compress the models parameters using a deep compression pipeline [13] only for client-to-server. In particular, we set the sparsity on 80% and 99% for the residuals in LetNet and CNN (5Conv + 3FC), respectively. We use an SGD optimizer momentum of 0.9. Those sparsified parameters are zero-parameters and the number of the continually appearing zero-parameters are encoded as a 16-bit float parameters [9]. After that, we quantize the non-zero parameters in 1 bit with median value of positive and negative parameters [6]. Finally, Huffman encoding [46] is used.

B. Experiments for Sec. VI-C

We train LeNet-5, CNN (5Conv+3FC) and ResNet18 (size from small to large) on Fashion-MNIST, CIFAR-10 and SVHN with 10 clients in both IID and Non-IID settings. We demonstrate the *ResFed* and lossy compress the residuals either only for uploading (UL) or downloading (DL) to study the effects on each direction separately. The learning rate is fixed on 0.01 and the batch size is 64 for all tests. In Non-IID data setting, each client owns 50% classes (5 out of 10). We use mean values from 5 tests in each experiment for the evaluation in Tab. III and show the results with standard variance in Fig. 6, which indicate that the overall communication volume can be remarkably reduced in *ResFed*.

For all experiments, we set the sparsity on 99% and use 1 bit for each non-zero parameters. Consequently, the compression ratio per communication round for LetNet-5 is approximately 350×, CNN (5Conv + 3FC) is approximately 375× and ResNet-18 is approximately 356×.

C. Experiments for Sec. VI-D

We train CNN (2Conv+1FC) for the experiments on FEMNIST [36]. We select the first 36 writers as clients. The data for each client are partitioned so that 80% is used for training and 20% for testing. A consistent learning rate of 0.01 is utilized across all methods. Except for FedSGD, all methods undergo 10 local epochs with a batch size of 1. For Top-K, we employ multiple sparsity levels in $\{80\%, 85\%, 90\%, 95\%, 99\%, 99.5\%\}$. Our goal is to identify the level that yields the highest comparison ratio. The best result is achieved with a sparsity of 80%. Other levels cannot efficiently reach a target accuracy of 70%. FTTQ does not reach the target accuracy 70%. For signSGD and QSFL, we refer to the results of [22], given that their experimental setup aligns with ours. The reported result for *ResFed* is based on a sparsity setting of 99.5%. Quantization uses 1 bit for all non-zero parameters.

D. Experiments for Sec. VI-E

To show the correlation between deep residual compression in up- und down-streaming, we train the CNN (5Conv + 3FC) model on IID CIFAR-10 with 10 clients and tune the sparsity to achieve different compression ratios per communication round in *ResFed*. The learning rate is fixed on 0.01 and the batch size is 64. We use an SGD optimizer momentum of 0.9. The number of local epochs is 1. Specifically, the value of sparsity is $\{0\%, 90\%, 95\%, 99\%, 99.5\%\}$ for both up- and down-streaming and then set 1 bit for all non-zero parameters in quantization.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/